SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1106

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

Assessment Tool Weightage: 60%

Dr Madhan V

QUESTIONS: 12 **Total Marks:60**

Annexure A

CONCEPT MAPPING

Code of Conduct:

I D Raghul (192511115) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CONCEPT MAPPING

1. Construct a Concept Map

Given:

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

Answer:

The missing concepts are:

- Design
- Deployment

Explanation:

- Design converts requirements into system architecture.
- Deployment releases the software to end users.

Analysis:

- Ensures systematic software development.
- Reduces project risks.
- Improves software quality and maintainability.

(5 Marks)

---

2. Construct a Concept Map

Given:

Object-Oriented Principles

→ Abstraction

→ _____

→ Inheritance

→ _____

Answer:

The missing concepts are:

- Encapsulation
- Polymorphism

Explanation:

- Encapsulation protects data from unauthorized access.
- Polymorphism allows one interface to support multiple behaviors.

Analysis:

- Improves security and flexibility.
- Enhances code reusability.
- Supports efficient software design.

(5 Marks)

---

3. Construct a Concept Map

Given:

Class → Blueprint

Object → Instance

Method → _____

Attribute → _____

Answer:

The missing concepts are:

- Behavior
- State

Explanation:

- Methods define the behavior of objects.
- Attributes represent the state of objects.

Analysis:

- Helps model real-world entities.
- Defines object characteristics and actions.
- Improves system organization.

(5 Marks)

---

4. A Concept Map Shows

Given:

Software Quality

→ Reliability

→ _____

→ Maintainability

Answer:

The missing concept is:

- Usability

Explanation:

- Usability measures how easily users interact with software.
- Reliable software performs consistently.
- Maintainable software is easy to update.

Analysis:

- Improves user satisfaction.
- Enhances product acceptance.
- Contributes to software success.

(5 Marks)

---

5. UML Concept Map

Given:

UML

→ Structural Diagrams → _____

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → _____

Answer:

The missing concepts are:

- Object Diagram
- Communication Diagram

Explanation:

- Object diagrams represent object instances and relationships.
- Communication diagrams show message exchange among objects.

Analysis:

- Improves system visualization.
- Supports object-oriented analysis.
- Facilitates stakeholder communication.

(5 Marks)

---

6. Given:

Requirements ✔ → Design ✔ → Coding ✖ → Testing ✖ → Deployment ✖

Explanation:

- Failure occurs during the Coding phase.
- Program logic contains implementation errors.

Analysis:

- Errors affect subsequent phases.
- Testing cannot proceed effectively.
- Software quality decreases.

Solution:

- Review source code.
- Apply coding standards.
- Conduct code inspections before testing.

(5 Marks)

7. Construct a Concept Map

Given:

Problem Identification → _____ → Design → Implementation → _____

Answer:

The missing concepts are:

- Requirement Analysis
- Testing

Explanation:

- Requirement Analysis identifies user needs and system requirements.
- Testing verifies whether the implemented system meets the specified requirements.

Analysis:

- Improves software quality.
- Reduces development errors.
- Ensures successful project completion.

(5 Marks)

---

8. Construct a Concept Map

Given:

Object-Oriented Features

→ Class

→ _____

→ Message Passing

→ _____

Answer:

The missing concepts are:

- Object
- Dynamic Binding

Explanation:

- Objects are instances of classes.
- Dynamic binding allows method calls to be resolved at runtime.

Analysis:

- Supports flexibility in software design.

- Improves code maintainability.
- Enables runtime polymorphism.

(5 Marks)

---

9. Construct a Concept Map

Given:

Software Development Models

→ Waterfall

→ _____

→ Spiral

→ _____

Answer:

The missing concepts are:

- Incremental Model
- Agile Model

Explanation:

- Incremental development delivers software in stages.
- Agile supports iterative development with continuous customer feedback.

Analysis:

- Enhances adaptability.
- Reduces project risks.
- Improves customer satisfaction.

(5 Marks)

---

10. Construct a Concept Map

Given:

Class Diagram

→ Class

→ _____

→ Association

→ _____

Answer:

The missing concepts are:

- Attribute
- Method

Explanation:

- Attributes define the properties of a class.
- Methods define the operations performed by a class.

Analysis:

- Represents object structure effectively.
- Improves software documentation.
- Supports object-oriented design.

(5 Marks)

---

11. Construct a Concept Map

Given:

Software Engineering Principles

→ Modularity

→ _____

→ Reusability

→ _____

Answer:

The missing concepts are:

- Maintainability
- Scalability

Explanation:

- Maintainability simplifies software updates.
- Scalability enables systems to handle increased workload efficiently.

Analysis:

- Improves software performance.
- Reduces maintenance effort.
- Supports long-term software evolution.

(5 Marks)

---

12. Given:

Requirement Analysis ✔ → System Design ✔ → Coding ✔ → Integration ✖ → Deployment ✖

Explanation:

- Failure occurs during the Integration phase.
- Individual modules fail to communicate correctly.

Analysis:

- Causes functional inconsistencies.
- Delays software deployment.
- Increases debugging effort.

Solution:

- Perform integration testing.
- Verify module interfaces.
- Resolve compatibility issues before deployment.

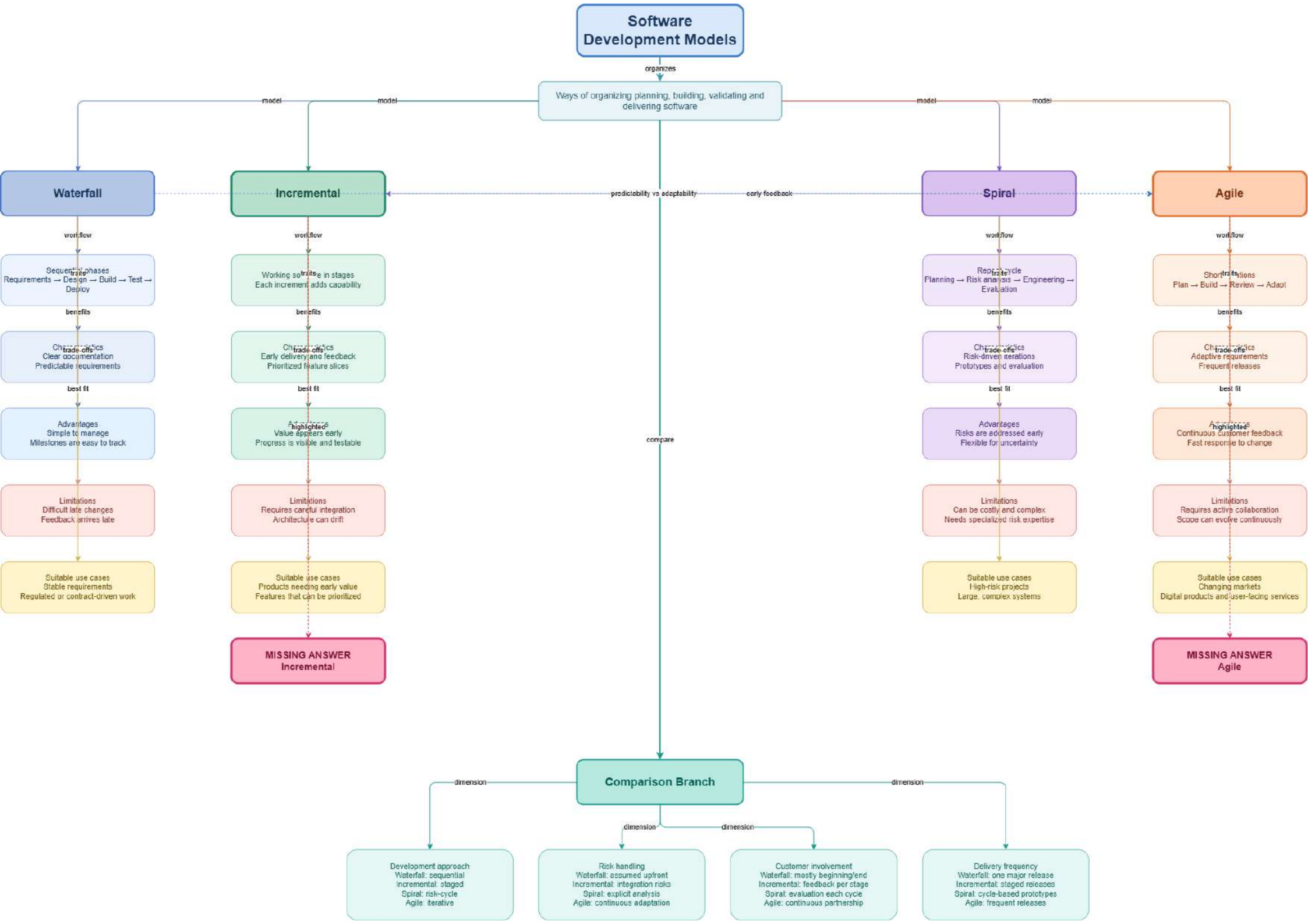
(5 Marks)

CONCEPT MAPPING

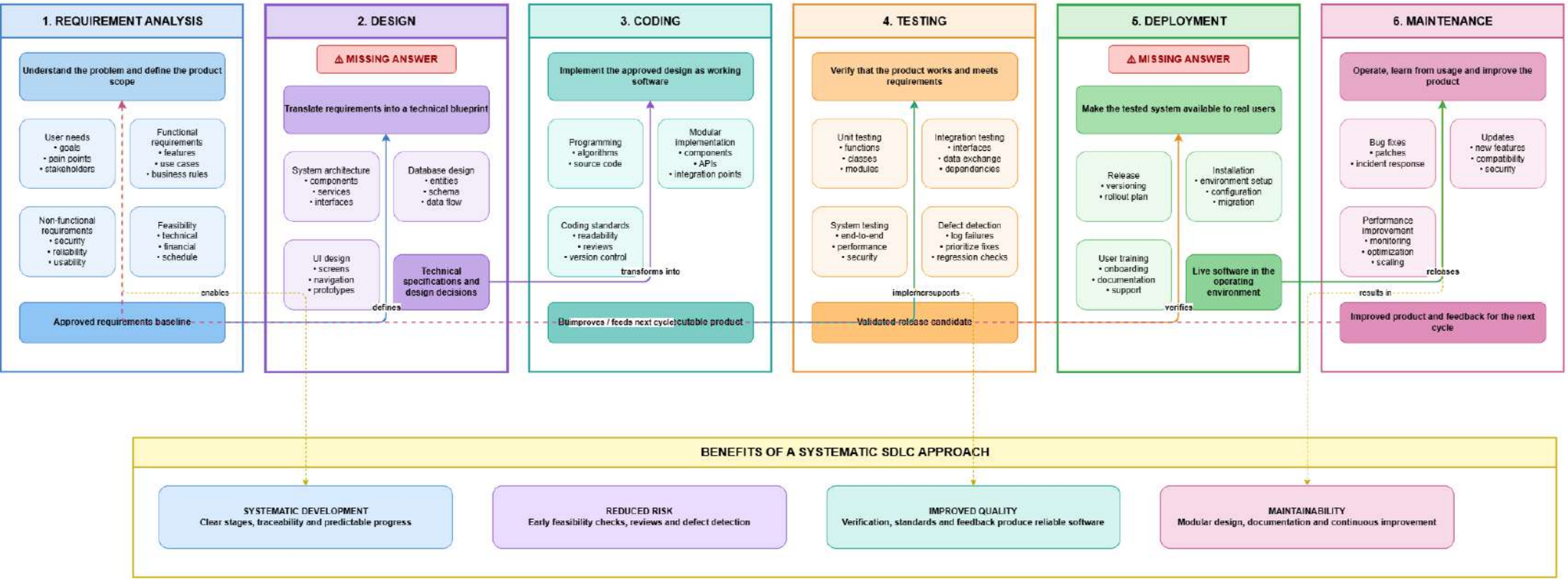
RUBRICS FOR CALCULATION

CONCEPT MAPPING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand


Software Development Models
organizes
Ways of organizing planning, building, validating and delivering software
model
model
model
model
Waterfall
Incremental
Spiral
Agile
predictability vs adaptability
early feedback
workflow
workflow
workflow
workflow
Sequential phases
Requirements → Design → Build → Test → Deploy
Working software in stages
Each increment adds capability
Repeat cycle
Planning → Risk analysis → Engineering → Evaluation
Short iterations
Plan → Build → Review → Adapt
traits
traits
traits
traits
benefits
benefits
benefits
benefits
Characteristics
Clear documentation
Predictable requirements
Characteristics
Early delivery and feedback
Prioritized feature slices
Characteristics
Risk-driven iterations
Prototypes and evaluation
Characteristics
Adaptive requirements
Frequent releases
trade-offs
trade-offs
trade-offs
trade-offs
best fit
best fit
best fit
best fit
Advantages
Simple to manage
Milestones are easy to track
Advantages
Value appears early
Progress is visible and testable
Advantages
Risks are addressed early
Flexible for uncertainty
Advantages
Continuous customer feedback
Fast response to change
highlighted
highlighted
compare
Limitations
Difficult late changes
Feedback arrives late
Limitations
Requires careful integration
Architecture can drift
Limitations
Can be costly and complex
Needs specialized risk expertise
Limitations
Requires active collaboration
Scope can evolve continuously
Suitable use cases
Stable requirements
Regulated or contract-driven work
Suitable use cases
Products needing early value
Features that can be prioritized
Suitable use cases
High-risk projects
Large, complex systems
Suitable use cases
Changing markets
Digital products and user-facing services
MISSING ANSWER
Incremental
MISSING ANSWER
Agile
Comparison Branch
dimension
dimension
dimension
dimension
Development approach
Waterfall: sequential
Incremental: staged
Spiral: risk-cycle
Agile: iterative
Risk handling
Waterfall: assumed upfront
Incremental: integration risks
Spiral: explicit analysis
Agile: continuous adaptation
Customer involvement
Waterfall: mostly beginning/end
Incremental: feedback per stage
Spiral: evaluation each cycle
Agile: continuous partnership
Delivery frequency
Waterfall: one major release
Incremental: staged releases
Spiral: cycle-based prototypes
Agile: frequent releases

SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) — CONCEPT MAP



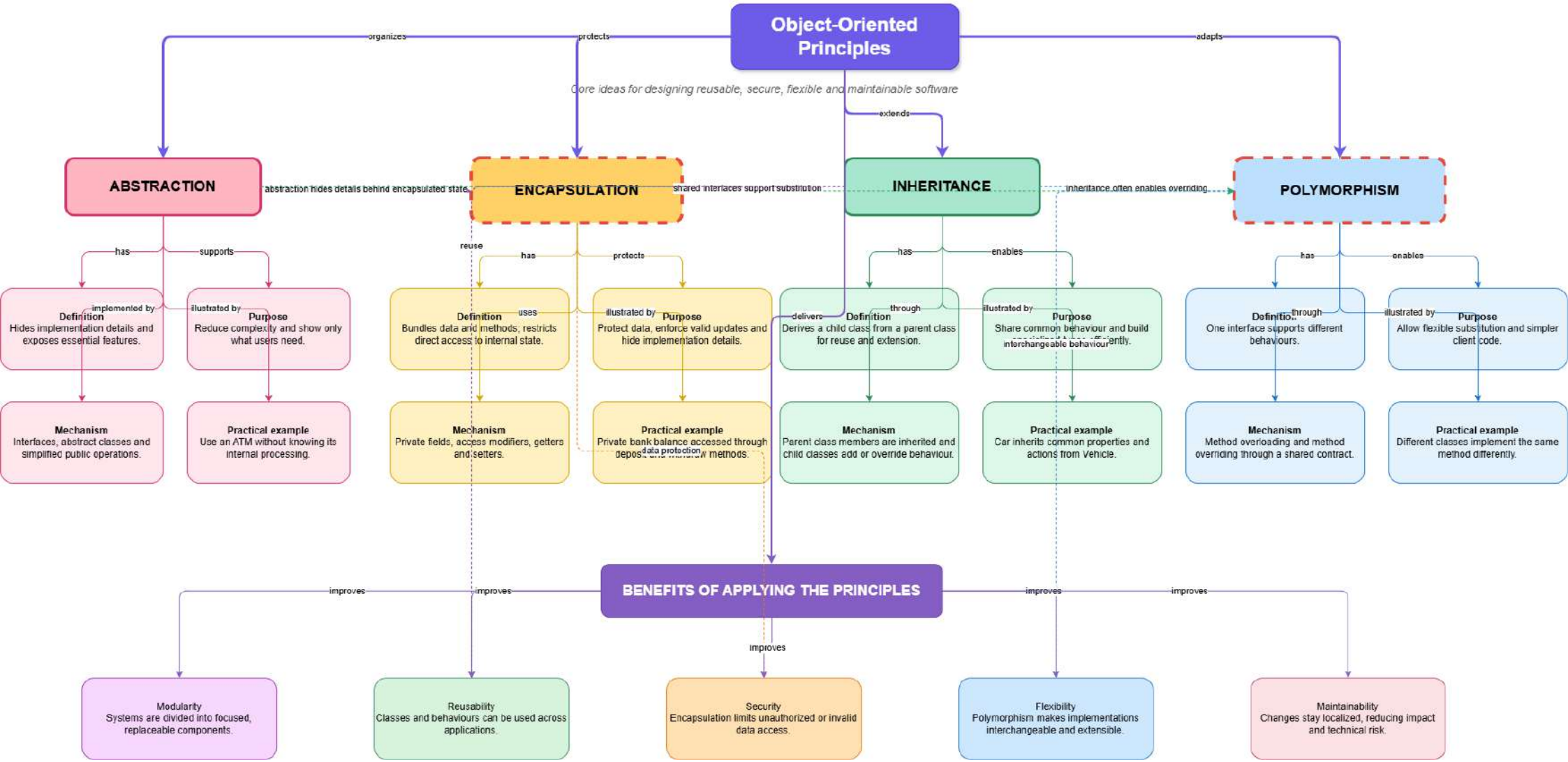
BENEFITS OF A SYSTEMATIC SDLC APPROACH

SYSTEMATIC DEVELOPMENT
Clear stages, traceability and predictable progress

REDUCED RISK
Early feasibility checks, reviews and defect detection

IMPROVED QUALITY
Verification, standards and feedback produce reliable software

MAINTAINABILITY
Modular design, documentation and continuous improvement


Object-Oriented Principles
Core ideas for designing reusable, secure, flexible and maintainable software
organizes
protects
extends
adapts
ABSTRACTION
abstraction hides details behind encapsulated state
ENCAPSULATION
shared interfaces support substitution
INHERITANCE
inheritance often enables overriding
POLYMORPHISM
has
supports
reuse
has
protects
has
enables
has
enables
implemented by
illustrated by
uses
illustrated by
delivers
through
illustrated by
through
illustrated by
Definition
Hides implementation details and exposes essential features.
Purpose
Reduce complexity and show only what users need.
Definition
Bundles data and methods, restricts direct access to internal state.
Purpose
Protect data, enforce valid updates and hide implementation details.
Definition
Derives a child class from a parent class for reuse and extension.
Purpose
Share common behaviour and build interchangeable behaviour efficiently.
Definition
One interface supports different behaviours.
Purpose
Allow flexible substitution and simpler client code.
Mechanism
Interfaces, abstract classes and simplified public operations.
Practical example
Use an ATM without knowing its internal processing.
Mechanism
Private fields, access modifiers, getters and setters.
Practical example
Private bank balance accessed through deposit and withdraw methods.
data protection
Mechanism
Parent class members are inherited and child classes add or override behaviour.
Practical example
Car inherits common properties and actions from Vehicle.
Mechanism
Method overloading and method overriding through a shared contract.
Practical example
Different classes implement the same method differently.
BENEFITS OF APPLYING THE PRINCIPLES
improves
improves
improves
improves
improves
Modularity
Systems are divided into focused, replaceable components.
Reusability
Classes and behaviours can be used across applications.
Security
Encapsulation limits unauthorized or invalid data access.
Flexibility
Polymorphism makes implementations interchangeable and extensible.
Maintainability
Changes stay localized, reducing impact and technical risk.